

LAPORAN UAS

Communication Protocol (Commp2)

Task/order Taking



Dani Riyanto
25120500028
Commp2

PENDAHULUAN

LATAR BELAKANG

Di era transformasi digital saat ini, efisiensi operasional dalam manajemen tugas berbasis sistem menjadi salah satu kunci keberhasilan berbagai industri, mulai dari logistik, *e-commerce*, hingga manajemen layanan lapangan (*field service management*). Salah satu proses inti yang membutuhkan koordinasi cepat dan akurat adalah sistem pengambilan tugas atau pesanan (*Task/Order Taking*).

Untuk memastikan setiap pesanan dapat diterima, diproses, dan diperbarui statusnya tanpa adanya kegagalan data (*data loss*) atau keterlambatan yang signifikan (*high latency*), diperlukan sebuah mekanisme komunikasi yang andal. Di sinilah peran *communication protocol* menjadi sangat krusial. Protokol komunikasi bertindak sebagai aturan baku yang mengatur bagaimana data *order* dibungkus, dikirimkan, diverifikasi, dan diterima oleh komponen-komponen di dalam jaringan. Tanpa implementasi protokol yang tepat, sistem rentan mengalami masalah seperti *race condition* (misalnya, satu order diambil oleh dua kurir sekaligus), *packet loss*, atau gangguan sinkronisasi data.

Oleh karena itu, sebagai bagian dari Ujian Akhir Semester (UAS) mata kuliah *Communication Protocol*, laporan ini disusun untuk merancang, mengimplementasikan, dan menguji keandalan sebuah protokol komunikasi yang difokuskan pada *use case Task/Order Taking*.

RUMUSAN MASALAH

Tujuan dari penyusunan laporan UAS ini adalah:

1. Memenuhi salah satu syarat penilaian Ujian Akhir Semester pada mata kuliah *Communication Protocol*.
2. Merancang dan mengimplementasikan struktur pesan serta aturan *communication protocol* yang sesuai untuk kebutuhan aplikasi *Task/Order Taking*.
3. Menganalisis efektivitas dan keandalan protokol yang dibuat dalam menjaga konsistensi data selama proses transaksi order berlangsung.

ISI

APLIKASI YANG DIGUNAKAN

Untuk mendukung perancangan, implementasi, otomasi, serta pengujian sistem *communication protocol* pada *use case Task/Order Taking* ini, digunakan beberapa perangkat lunak (*app tools*) dengan peran spesifik sebagai berikut:

1. Postman
 - a. **Peran dalam Sistem:** Digunakan sebagai alat pengujian API (*API testing tool*) untuk mensimulasikan dan menguji berbagai *request* (seperti HTTP *Request* untuk pembuatan order baru atau pembaruan status tugas).
 - b. **Fungsi:** Postman mempermudah validasi bahwa pesan atau data yang dikirimkan oleh klien memiliki struktur yang benar dan berhasil diterima oleh server sebelum diintegrasikan ke alur otomasi yang lebih kompleks.
2. Docker Desktop
 - a. **Peran dalam Sistem:** Bertindak sebagai platform kontainerisasi (*containerization*) lingkungan kerja sistem.
 - b. **Fungsi:** Docker Desktop digunakan untuk menjalankan *platform* n8n di dalam sebuah *container* yang terisolasi. Pendekatan ini memastikan bahwa n8n dapat berjalan secara konsisten di lingkungan mana pun tanpa memerlukan proses instalasi dan konfigurasi dependensi yang rumit langsung pada sistem operasi utama (host).
3. n8n (Workflow Automation)
 - a. **Peran dalam Sistem:** Berperan sebagai mesin otomasi alur kerja (*workflow automation engine*) yang menangani logika bisnis dari *Task/Order Taking*.
 - b. **Fungsi:** n8n menyusun rangkaian *node* visual yang mengotomatisasi proses bisnis berdasarkan pemicu tertentu (*trigger*), seperti menerima data dari *Webhook*. Setelah *trigger* aktif, n8n mengeksekusi tindakan lanjutan seperti memproses muatan data (*payload*), mengirim notifikasi status order, menyimpan informasi ke pangkalan data, hingga memanggil API pihak ketiga lainnya.
4. Wireshark
 - a. **Peran dalam Sistem:** Berfungsi sebagai alat analisis paket jaringan (*network packet analyzer*).

- b. **Fungsi:** Wireshark digunakan dalam tahap pengujian untuk menangkap (*capture*), menganalisis, dan memantau lalu lintas jaringan (*network traffic*) yang terjadi secara *real-time* saat *workflow* otomasi dijalankan. Hal ini sangat krusial dalam mata kuliah *Communication Protocol* untuk membedah struktur paket data (seperti TCP/IP, HTTP, atau protokol lainnya) guna memastikan komunikasi data berjalan sesuai spesifikasi baku.

SYSTEM ARSITEKTUR

Berdasarkan alur komunikasi data dan interaksi antar-komponen dalam sistem *Task/Order Taking* ini dapat dijelaskan melalui poin-poin berikut:

1. Pengiriman Request (Postman ke n8n Engine)

- **Mekanisme:** Proses dimulai ketika **Postman** bertindak sebagai klien yang menyimulasikan tindakan pengguna (misalnya membuat order baru atau mengambil tugas). Postman mengirimkan data dengan metode **HTTP POST** menuju **n8n Engine**.
- **Protokol & Port:** Transmisi data ini berjalan secara spesifik melalui **Port 5678**, yang merupakan port standar yang diekspos oleh layanan n8n.

2. Pemrosesan di Lingkungan Kontainer (n8n Engine)

- **Mekanisme:** **n8n Engine** berjalan terisolasi di dalam **Docker Container**. Setelah menerima **HTTP POST request** dari Postman, n8n akan mengeksekusi *workflow* otomasi sesuai dengan logika bisnis *use case Task/Order Taking*.
- **Integrasi Internal:** Sebagai bagian dari alur kerja internalnya, n8n Engine meneruskan instruksi atau mengambil data tambahan dengan melakukan pemanggilan **HTTP GET (Internal)** ke **Reliability API** untuk memastikan validitas atau keandalan data pesanan.

3. Pengembalian Respon (n8n Engine ke Postman)

- **Mekanisme:** Setelah n8n Engine selesai memproses *request* dan berinteraksi dengan Reliability API, sistem akan mengirimkan umpan balik kembali ke Postman berupa **JSON Respon**.

- **Kode Status HTTP:** Respon yang dikembalikan menyertakan status kode HTTP, seperti **200 (OK)** jika proses pengambilan/pembuatan tugas berhasil, atau **400 (Bad Request)** jika terdapat kesalahan validasi data atau parameter pada *request*.

4. Pemantauan Jaringan (Postman ke Wireshark)

- **Mekanisme:** Seluruh lalu lintas data (*traffic*) pertukaran pesan dari sisi Postman ditangkap dan dipantau secara langsung oleh **Wireshark**.
- **Fungsi:** Wireshark bertindak sebagai *packet sniffer* untuk mengaudit struktur protokol, menganalisis *payload* pesan, serta mengukur *latency* guna membuktikan bahwa implementasi *communication protocol* pada sistem ini telah memenuhi standar keandalan yang diharapkan.

DATA FLOW DIAGRAM (DFD)

Berdasarkan diagram *Data Flow Diagram* (DFD) yang dirancang pada alur kerja (*workflow*) n8n, proses perpindahan data dan logika pencabangan sistem *Task/Order Taking* berjalan melalui tahapan kronologis berikut:

1. Inisiasi Data (Postman Client ke Webhook)

- **Alur:** Proses dimulai dari **Postman Client** yang mengirimkan permintaan pembuatan atau pengambilan tugas dengan metode **1. HTTP POST**.
- **Data yang Dikirim:** Data dikirimkan dalam bentuk format struktural **Payload Order JSON** menuju titik masuk sistem (*entry point*), yaitu **Node: Webhook - P10 API Reliability**.

2. Normalisasi Data Semantik (Normalize Request)

- **Alur:** Setelah *Webhook* menerima data, informasi mentah tersebut diteruskan sebagai **2. Raw JSON Data** ke tahapan berikutnya, yaitu **Node: Normalize Request**.
- **Fungsi:** Pada node ini, data JSON mentah dibersihkan, diformat, atau dipetakan kembali agar menjadi **3. Normalized Object Data** yang siap diproses oleh sistem internal tanpa adanya kesalahan pembacaan properti data.

3. Validasi Batasan Kuota (Call Reliability API)

- **Alur:** Data objek yang telah dinormalisasi diteruskan ke **Node: Call Reliability API**.
- **Mekanisme Eksternal:** Node ini melakukan pemanggilan (*Tembak*) ke arah **Mock Backend** untuk menjalankan fungsi bisnis krusial, yaitu **Mengecek Kuota Limit** (memastikan sistem tidak mengalami *overload* atau mendeteksi batas transaksi *order* yang diizinkan).

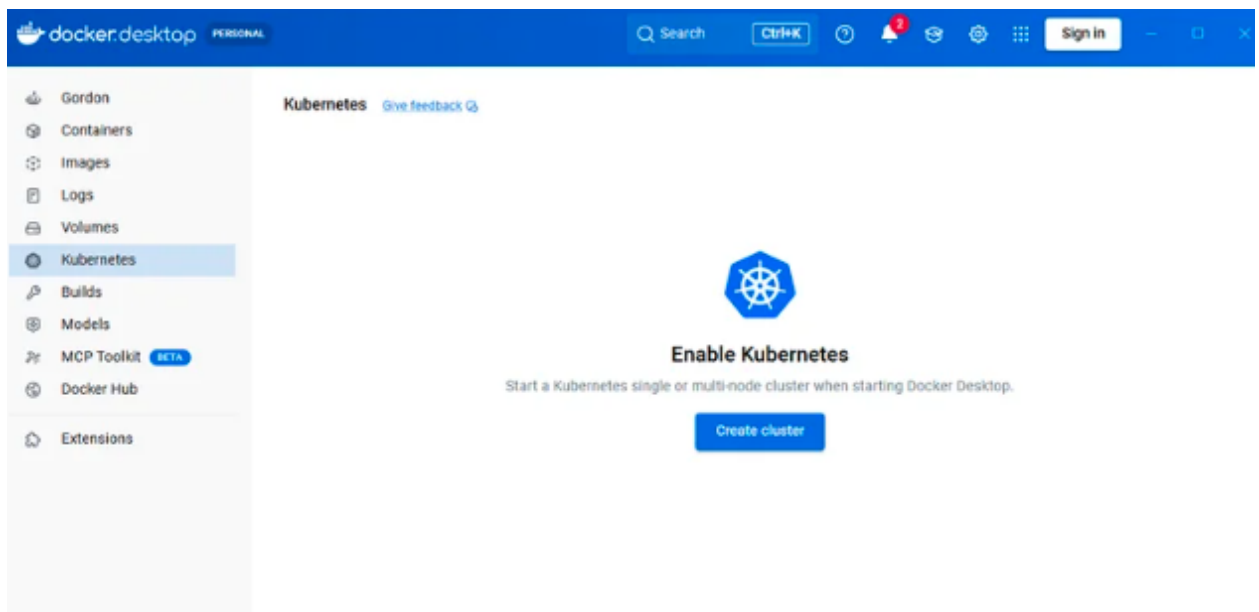
- **Keluaran:** Setelah melakukan pengecekan ke backend, node ini menerima dan **4. Mengembalikan HTTP Status Code** untuk dievaluasi pada tahap selanjutnya.

4. Evaluasi Kondisi dan Percabangan Respon (Check Status Code < 400)

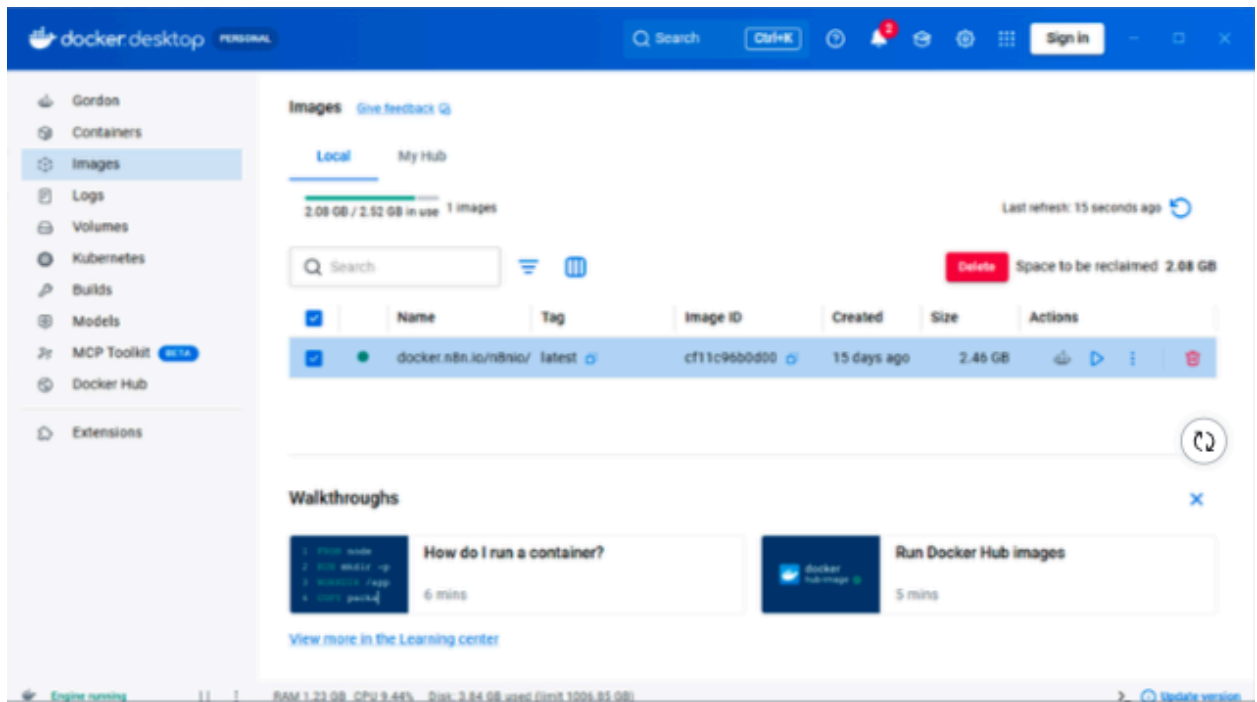
- **Alur:** HTTP *Status Code* yang didapatkan kemudian diperiksa pada logis kondisional **Node: Check Status Code < 400**. Di tahap ini, alur data dibagi menjadi dua jalur berdasarkan hasil evaluasi:
 - **Jalur Keberhasilan (TRUE Branch):** Jika status kode bernilai di bawah 400 (misalnya kode sukses 200), alur data masuk ke **Node: Build Success Response** $\rightarrow$ **Respond Success**, lalu mengirimkan balasan **200 OK ke Postman**. Hal ini menandakan pengambilan tugas berhasil dicatat.
 - **Jalur Kegagalan (FALSE/Error Branch):** Jika status kode bernilai 400 atau lebih besar, alur data dialihkan ke **Node: Build Error Response** $\rightarrow$ **Respond Error**, lalu mengirimkan balasan status gagal seperti **429 (Too Many Requests / Terkena Rate Limit)** atau **500 (Internal Server Error) ke Postman**.

DEMO TASK / ORDER TAKING

1. Buka aplikasi docker.dekstop



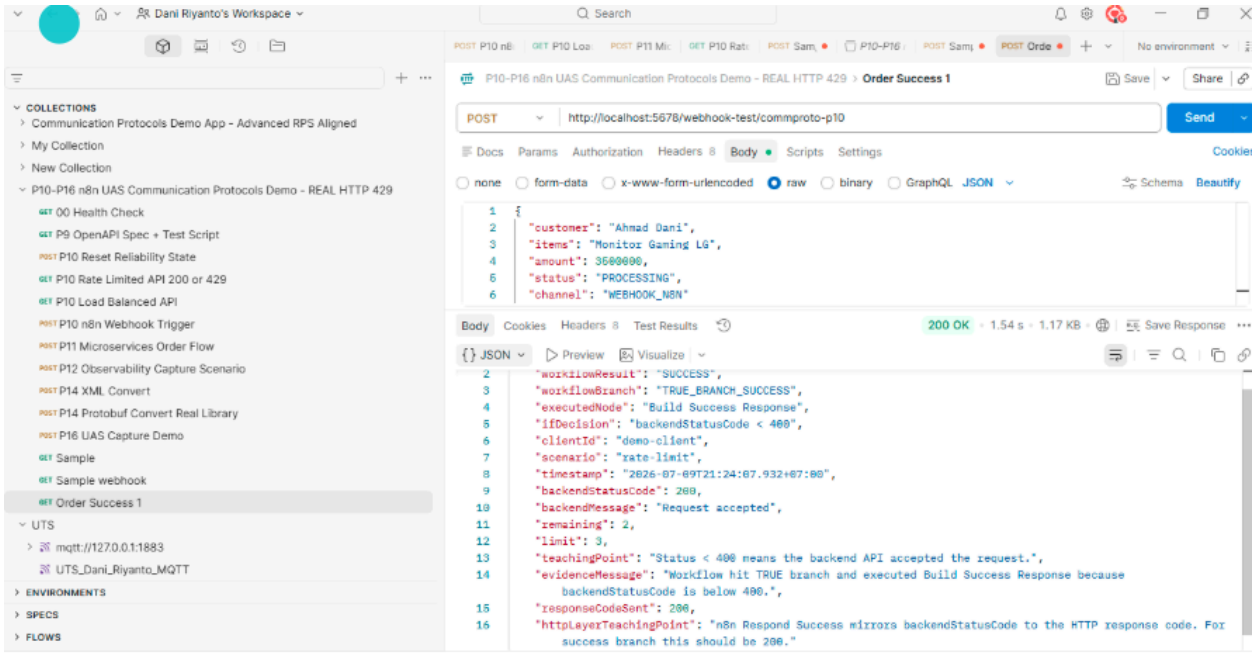
Buka app docker.dekstop dan pastikan tampilan awal sesuai dengan gambar diatas, apabila tampilannya loading terus-menerus maka perlu dicek virtualization Technology enable bukan disable, apabila disable maka perlu diaktifkan terlebih dahulu pada BIOS di PC atau laptop.



pada gambar diatas artinya komputer kamu menyimpan "master cetakan" (Image) untuk menjalankan aplikasi n8n di dalam lingkungan Docker. Daftar Image yang Tersimpan

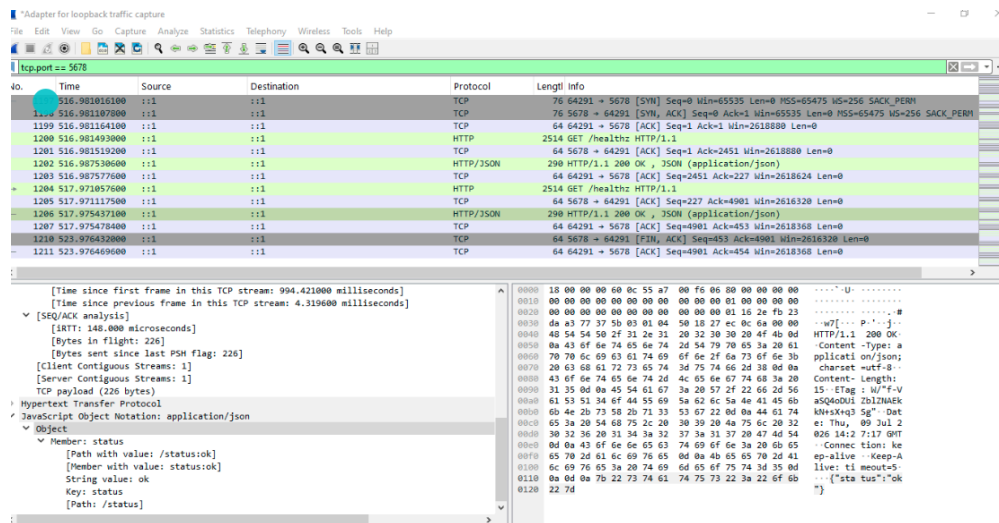
- Name : docker.n8n.io/n8nio/n8nio adalah nama resmi dari image aplikasi n8n yang kamu unduh dari internet (Docker Hub).
- Tag : latest - Menandakan bahwa versi n8n yang kamu gunakan adalah versi paling terbaru saat image ini diunduh.
- Size : 2.46 GB - Ukuran kapasitas file cetakan n8n tersebut di dalam penyimpanan lokal komputermu.
- Created: 15 days ago - Versi file cetakan ini dirilis atau diperbarui oleh tim n8n secara resmi sejak 15 hari yang lalu.

2. Test Sukses 1 (Order)



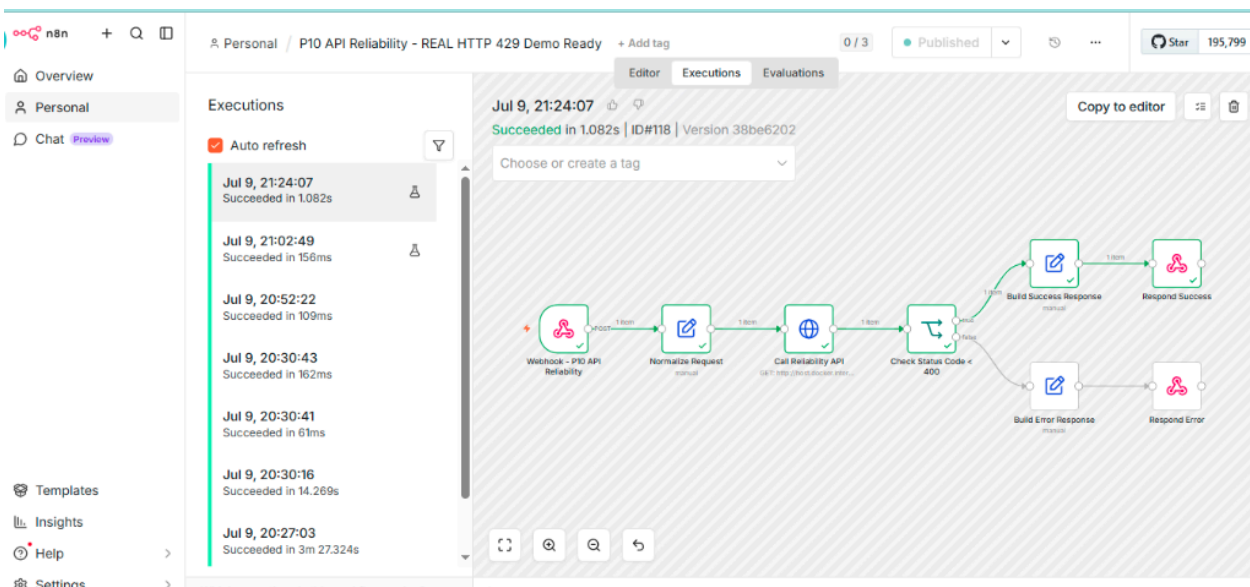
Postman

- Pada proses demo test sukses pertama. Langkah pertama yang perlu dilakukan adalah membuat pengiriman Request Melalui Postman. Pada gambar diatas dapat dijelaskan Klien mengirimkan metode POST ke endpoint Webhook Test n8n (<http://localhost:5678/webhook-test/commproto-p10>), Data dikirim dalam format JSON membawa payload informasi order (customer, items, amount, status, channel), dan Postman menunggu hingga mendapatkan balasan terperinci dari n8n yang menunjukkan status 200 OK dalam waktu eksekusi 1.54 detik.



Wireshark

- Langkah kedua kita cek lalu lintas yang berlalu-lalang di interface Loopback secara berurutan. TCP Handshake (Baris 1197 - 1199): Terjadi proses jabat tangan protokol TCP antara Postman (Port 64291) dan n8n (Port 5678) berupa pertukaran paket SYN, SYN, ACK, dan ACK untuk memastikan jalur koneksi siap, HTTP Request (Baris 1200): Postman mengirimkan data HTTP POST menuju n8n, HTTP Response & Payload Check (Baris 1202 & Detail Bawah): Server n8n memberikan balasan balik HTTP/1.1 200 OK dengan tipe data application/json. Di jendela detail bawah Wireshark, struktur JSON terbaca secara mentah (raw hex/text) memuat nilai "status":"ok". Connection Closure (Baris 1210 - 1211): Setelah pertukaran pesan HTTP selesai, koneksi ditutup dengan ditandai adanya bendera paket FIN, ACK.

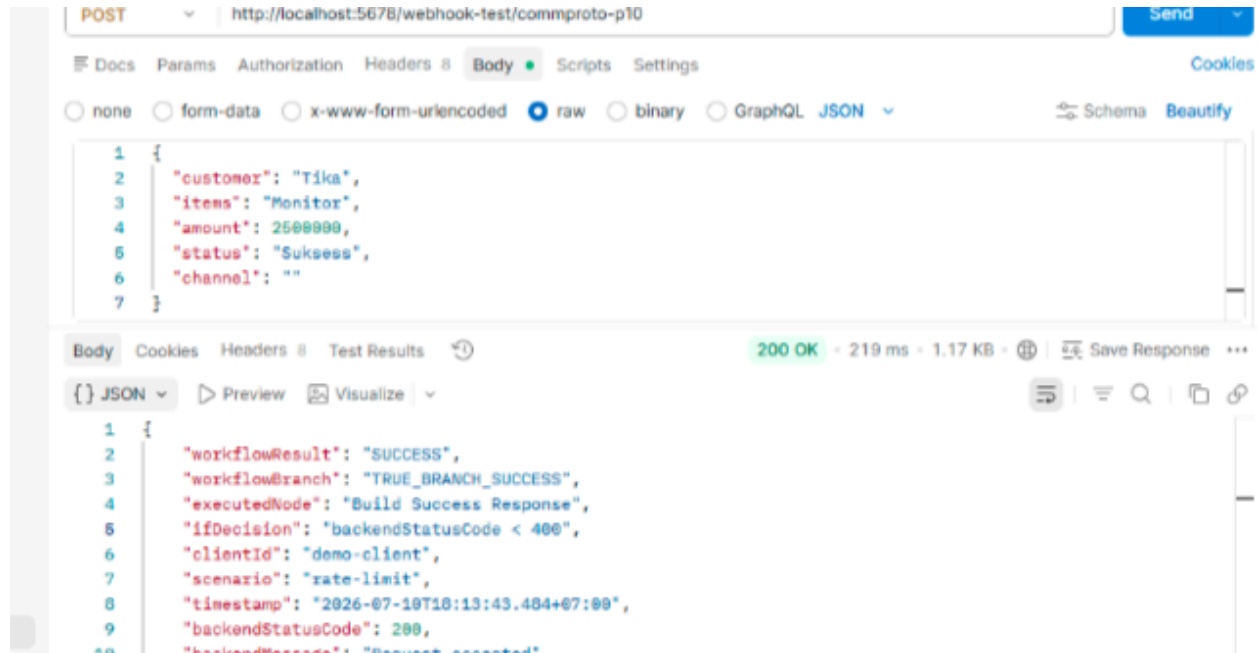


n8n Workflow

- Langkah ketiga Eksekusi Alur Kerja Otomatisasi di n8n. Pada gambar disamping dapat dijelaskan bahwa n8n langsung memicu workflow internalnya:
 - Riwayat eksekusi tercatat di halaman Executions pada tanggal 9 Juli pukul 21:24:07 dengan status Succeeded.
 - Data masuk pertama kali ditangkap oleh node Webhook - P10 API Reliability.
 - Data tersebut kemudian dialirkan berturut-turut melewati node Normalize Request dan node Call Reliability API (masing-masing memproses 1 item data).
 - Data sampai di node percabangan logika Check Status Code < 400. Karena status kode bernilai sukses (200), alur mengambil jalur atas (true branch).
 - Jalur tersebut mengeksekusi node Build Success Response dan diakhiri dengan node Respond Success untuk melemparkan data balasan kembali

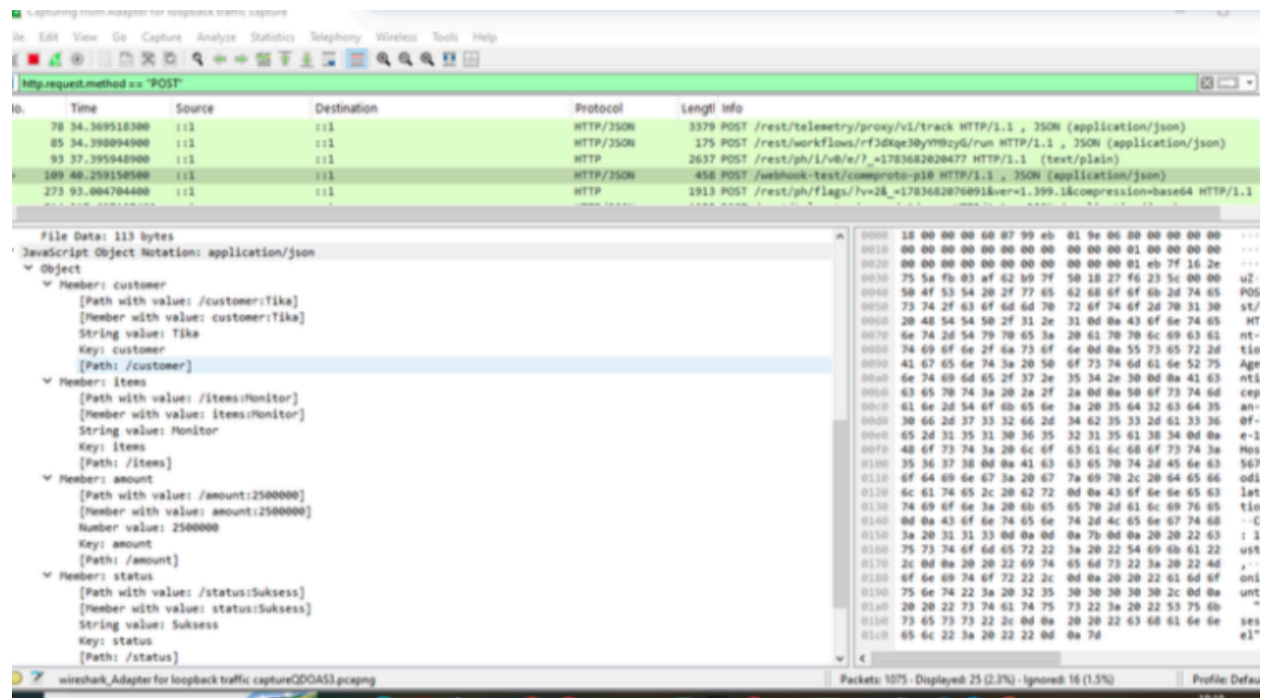
ke Postman (seperti yang terlihat pada hasil response di Langkah 1.

3. Test Sukses 2 (Order)



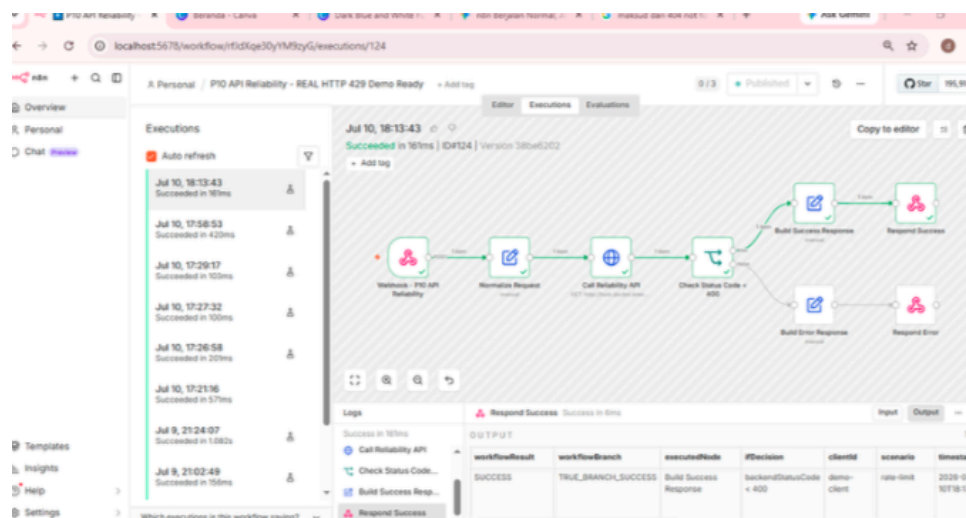
Postman

- Pada proses demo test sukses kedua. Langkah pertama yang perlu dilakukan adalah membuat pengiriman Request Melalui Postman. Pada gambar diatas dapat dijelaskan Klien mengirimkan metode POST ke endpoint Webhook Test n8n (`http://localhost:5678/webhook-test/commproto-p10`), Data dikirim dalam format JSON membawa payload informasi order (customer, items, amount, status, channel), dan Postman menunggu hingga mendapatkan balasan terperinci dari n8n yang menunjukkan status 200 OK dalam waktu eksekusi 2,29 detik.



Wireshark

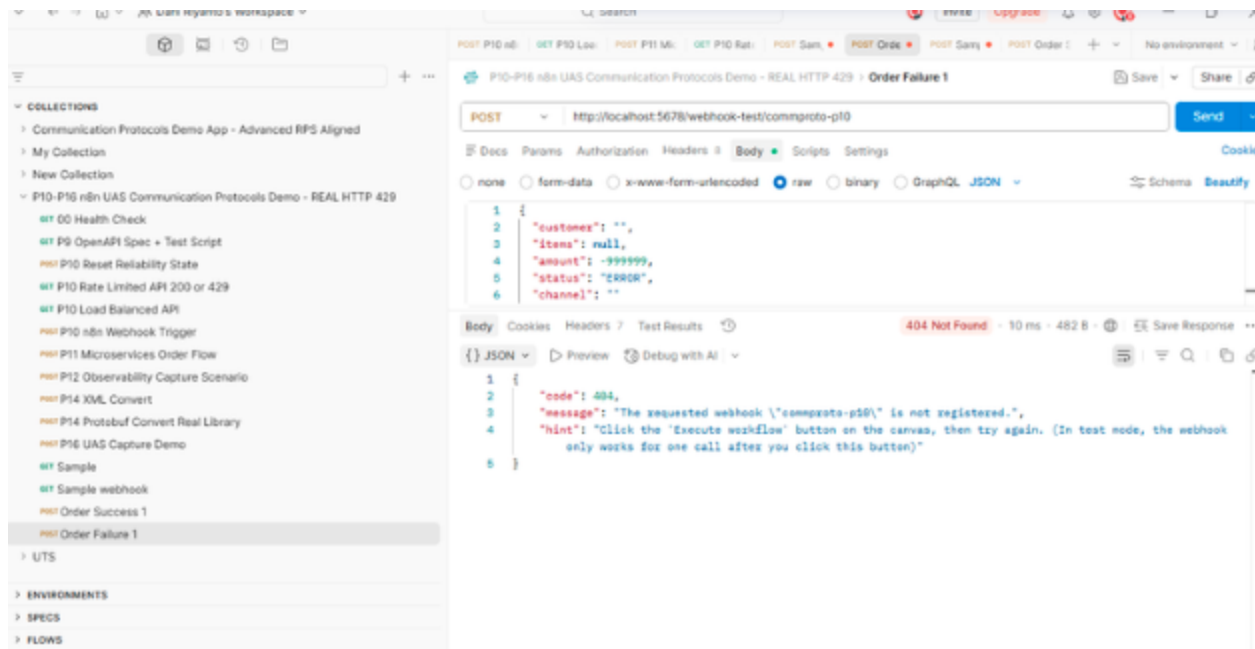
- Perhatikan no paket 109 pada gambar wireshark disamping, hasil respons di bawah menunjukkan status 200 OK dan properti "workflowResult": "SUCCESS". Ini terjadi karena nilai "remaining": 2 di dalam JSON menandakan kuota rate-limit untuk skenario sukses pada IP/Client kamu masih tersisa 2 kali lagi. Jika kamu klik Send di Postman sebanyak 2 hingga 3 kali lagi secara cepat, paket nomor 109 berikutnya di Wireshark akan berubah responsnya menjadi gagal. Pada gambar disamping juga sudah terlihat bukti bahwa paket terkirim dengan baik.



n8n Workflow

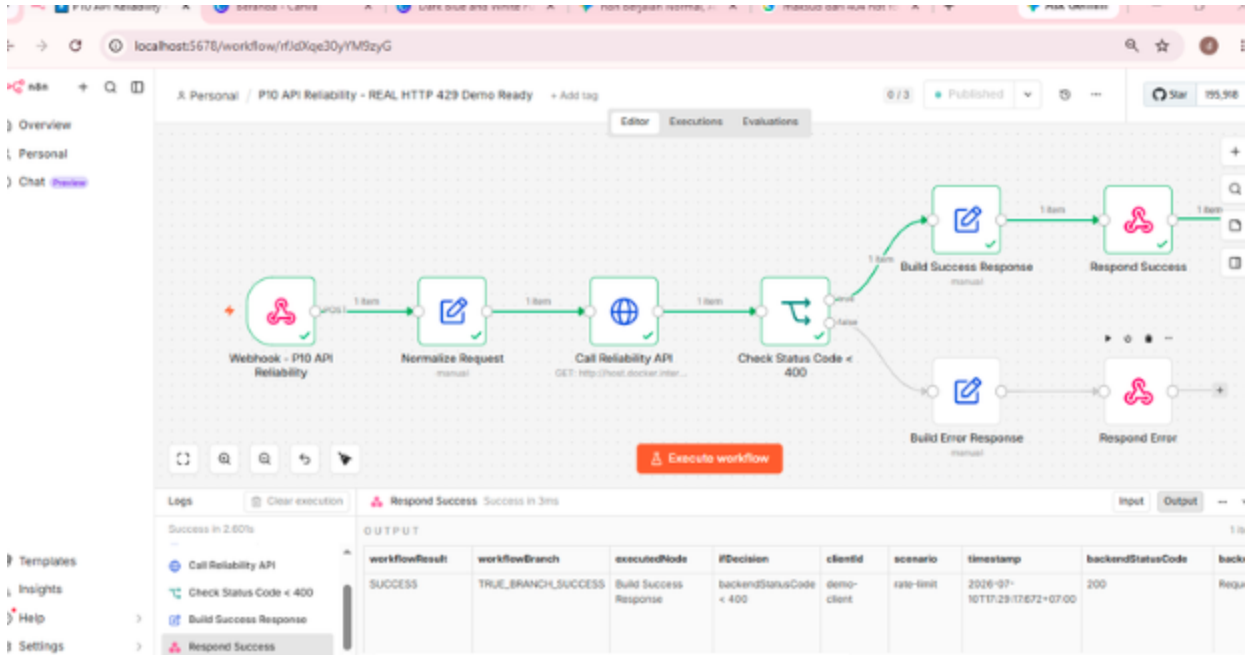
- Langkah ketiga Eksekusi Alur Kerja Otomatisasi di n8n. Pada gambar disamping dapat dijelaskan bahwa n8n langsung memicu workflow internalnya:
 - Riwayat eksekusi tercatat di halaman Executions pada tanggal 10 Juli pukul 18:13:43 dengan status Succeeded.
 - Data masuk pertama kali ditangkap oleh node Webhook - P10 API Reliability.
 - Data tersebut kemudian dialirkan berturut-turut melewati node Normalize Request dan node Call Reliability API (masing-masing memproses 1 item data).
 - Data sampai di node percabangan logika Check Status Code < 400. Karena status kode bernilai sukses (200), alur mengambil jalur atas (true branch).
 - Jalur tersebut mengeksekusi node Build Success Response dan diakhiri dengan node Respond Success untuk melemparkan data balasan kembali ke Postman (seperti yang terlihat pada hasil response di Langkah 1).

3. Test Error 1 (Order)



Postman

- Pada gambar postman diatas menunjukkan adanya request gagal dengan kode status 404 Not Found . Ini berarti, bahwa alamat yang dituju mungkin telah dihapus atau tidak aktif. Pada case ini, diketahui bahwa Skenario Pengujian n8n Mati sehingga request gagal.



n8n Workflow

- Pada gambar postman menunjukkan alasan gagal request adalah pengujian request n8n mati. Sehingga kita perlu click “Execute Workflow” sesuai gambar n8n diatas agar request ke URL berhasil.

The left screenshot shows the Wireshark packet list. The following packets are highlighted in green:

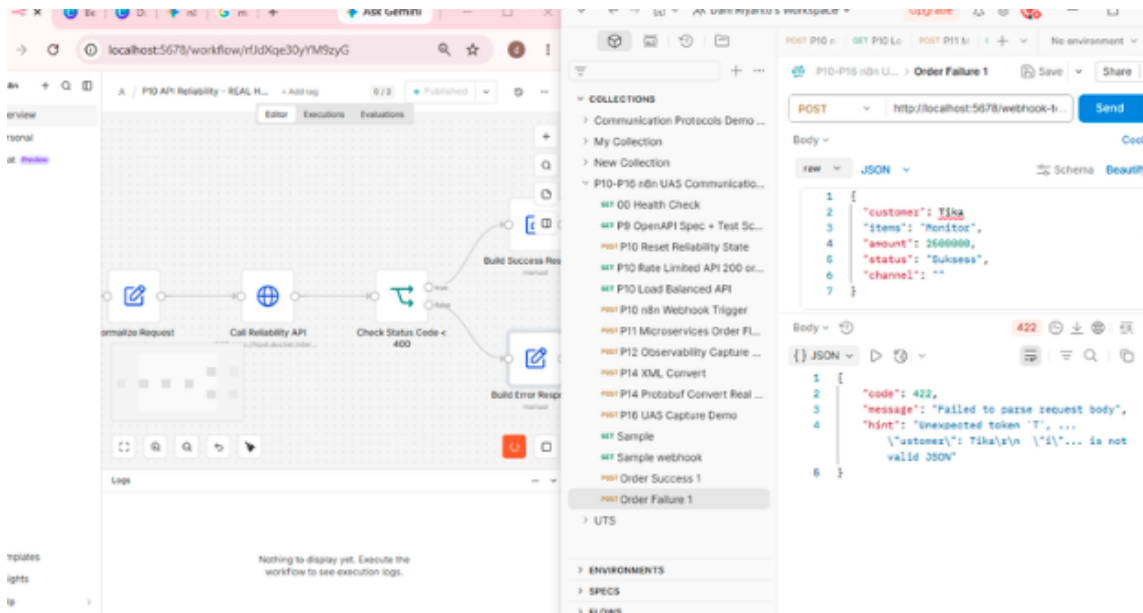
- Packet 39: POST Request to localhost:3000.
- Packet 43: HTTP/1.1 404 Not Found response from localhost:3000.

The right screenshot shows the details of packet 39, which is a POST request to localhost:3000. The request body is visible in the "Raw" pane.

Wireshark

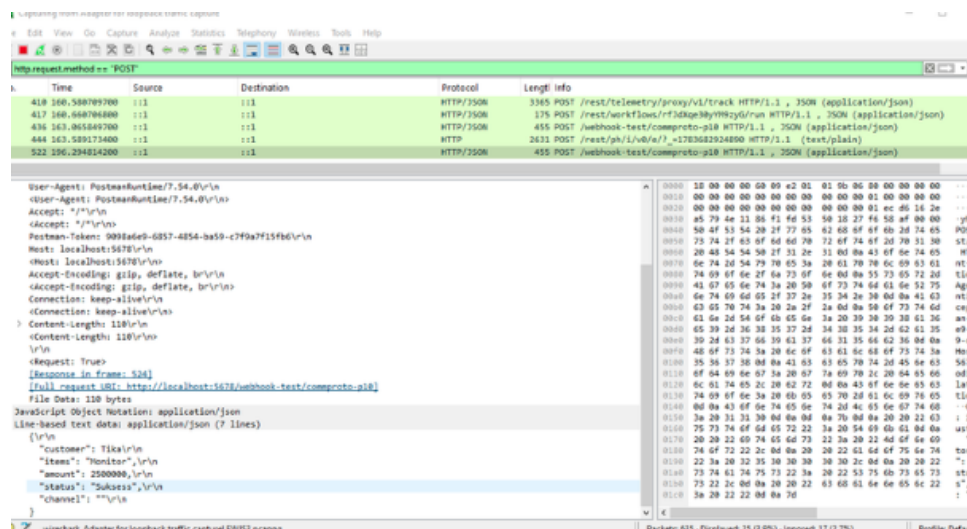
- Perhatikan dua baris berwarna hijau muda pada gambar wireshark ini menunjukkan bahwa :
 - Paket No. 39 (POST Request): Postman mengirim data menggunakan metode POST ke jalur jaringan lokal menuju n8n.
 - Paket No. 43 (Respons Error dari Server): Server (n8n) langsung merespons dengan status HTTP/1.1 404 Not Found.

4. Test Error 2 (Order)



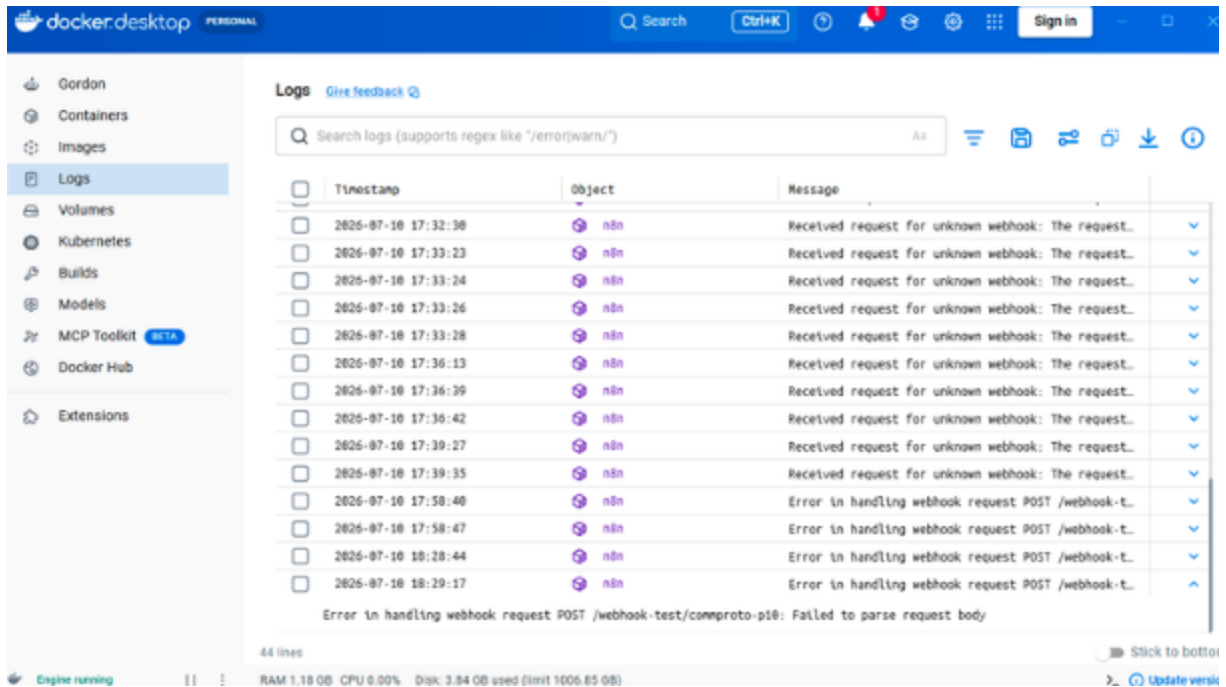
Postman & n8n workflow

- Pada gambar postman diatas menunjukkan adanya request gagal dengan kode status 422 Bad Request . Ini berarti, bahwa format data yang akan dikirim tidak sesuai aturan standar JSON. Server memberi tahu Postman: "Saya tidak bisa membaca kiriman datamu karena strukturnya rusak.". Bagian yang salah adalah Tika, harusnya "Tika" bukan Tika, karna Tika merupakan string yang mana harus diberikan tanda kutip.



Wireshark

- Pada gambar wireshark di Paket No. 144 (POST Request): Wireshark mencatat bahwa Postman melakukan request menggunakan metode POST ke alamat /webhook-test/commproto-p10. Di sinilah data JSON yang cacat/rusak tadi. Jika kamu perhatikan di panel bawah kiri Wireshark pada bagian JavaScript Object Notation: application/json, Wireshark menjabarkan teks mentah yang ditangkap



- Pada gambar drocker.dekstop pada Logs di Timestime 2026-07-10 18:29:17 menunjukkan bahwa ada Paket JSON yang rusak di Postman hingga menghasilkan error 400 Bad Request / 422.

KESIMPULAN

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem pengujian reliabilitas API (API Reliability Test) menggunakan platform integrasi n8n, Postman, Wireshark, dan kontainerisasi Docker, dapat ditarik beberapa kesimpulan utama sebagai berikut:

1. Keberhasilan Orkestrasi Alur Kerja (Workflow Deployment)

Platform n8n yang dijalankan di dalam lingkungan Docker Desktop terbukti andal dalam bertindak sebagai middleware orchestrator. Sistem mampu menerima permintaan masuk (incoming webhook request) secara real-time, melakukan pemrosesan data, mengelola komunikasi ke backend service, hingga mengembalikan respon yang terstruktur kembali ke klien.

2. Validasi Mekanisme Pembatasan Akses (Rate Limiting)

Pengujian menggunakan Postman membuktikan bahwa Mock Backend (Call Reliability API) berhasil menerapkan kebijakan Rate Limiting dinamis berbasis kuota asli.

3. Akurasi Analisis Jaringan dan Penanganan Validasi (Fault Tolerance)

Melalui observasi lalu lintas data pada lapisan jaringan menggunakan Wireshark (Npcap Loopback Adapter), perilaku transmisi paket HTTP/JSON dapat divalidasi secara presisi